TRIC MindSpark Series: ROS2 System Integration

Practical Worksheet 01 – Workspace, Python Package & First Node

Objective

Create a ROS2 workspace and Python package, develop a custom node, build the package, and verify runtime execution.

Module: ROS2 System Development from Scratch

Platform: Ubuntu 24.04 LTS

Platform: Ubuntu 24.04 LTS

Middleware: ROS2 Jazzy

Language: Python 3

Estimated Duration: 60–75 minutes

Prerequisites Check: ROS2 Jazzy Desktop and **ros-dev-tools** installed.

⚠️ (Note: OpenCV, ESP32, or PySerial setup are explicitly **NOT** required for PW1).

Gemini Notebook

The ROS2 Architecture at a Glance

An exploded isometric visualization of the hierarchical containers defining ROS2 software development and execution.

Development Phase

The Workspace (Source & Build)

```
src/     ~/ros2_ws/src/my_package
build/   ~/ros2_ws/src/my_package
install/ ~/ros2_ws/src/my_package
```

Compilation & Linking

The Package (Distributable Unit)

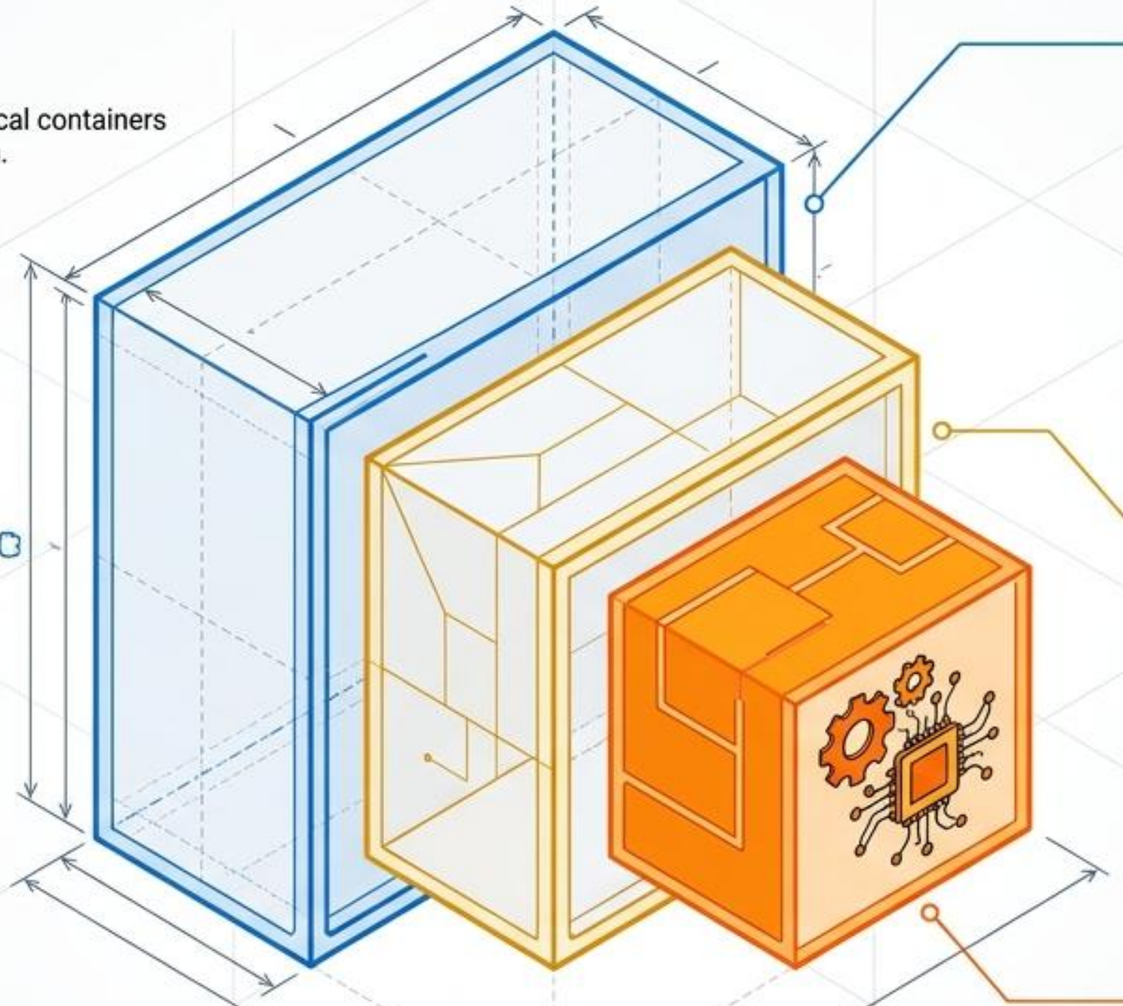
```
- CMakeLists.txt
- package.xml
- launch/
- config/
```

Execution & Launch

The Node (Runtime Process)

```
ros2 run my_package my_node
- Active data publishing/subscribing
- Parameter handling
- Service calls troup my_node
```

Note: Nodes communicate via Topics, Services, and Actions, facilitating modular system design.



The Workspace

The overarching container for building and isolating custom projects.

The Package

The distributable unit of ROS2 code containing metadata and dependencies.

The Node

The active, running executable process that performs actual computation.

exe

TRIC
TECHNOLOGY OF ROBOTICS AND IOT CENTRE

Gemini Notebook

Task 1: Environment Setup & Verification

```
$ source /opt/ros/jazzy/setup.bash

$ echo $ROS_DISTRO

$ ros2 pkg list | grep demo_nodes_py
```

Expected Output: 'jazzy' and 'demo_nodes_py' are displayed.

The Environment Matrix: Underlay vs. Overlay

Global Underlay	Local Overlay
- Path: /opt/ros/jazzy/...	- Path: ~/mindspark_ws/install/...
- Contains the core ROS2 installation.	- Contains your custom written code.
- Provided by the system; strictly Read-Only.	- Built by you; Read-Write capabilities.
- Must be sourced FIRST.	- Must be sourced SECOND to override/append.

Gemini Notebook

Task 2: The Workspace & Package Paradigm

```
$ mkdir -p ~/mindspark_ws/src
$ cd ~/mindspark_ws/src
$ ros2 pkg create mindspark_robot --build-type ament_python \
   --dependencies rclpy std_msgs geometry_msgs
```

Tells Colcon to treat this as a Python package, generating a `setup.py` instead of a CMakeLists.txt.

Automatically injects these core ROS2 message and Python client libraries directly into your `package.xml`.



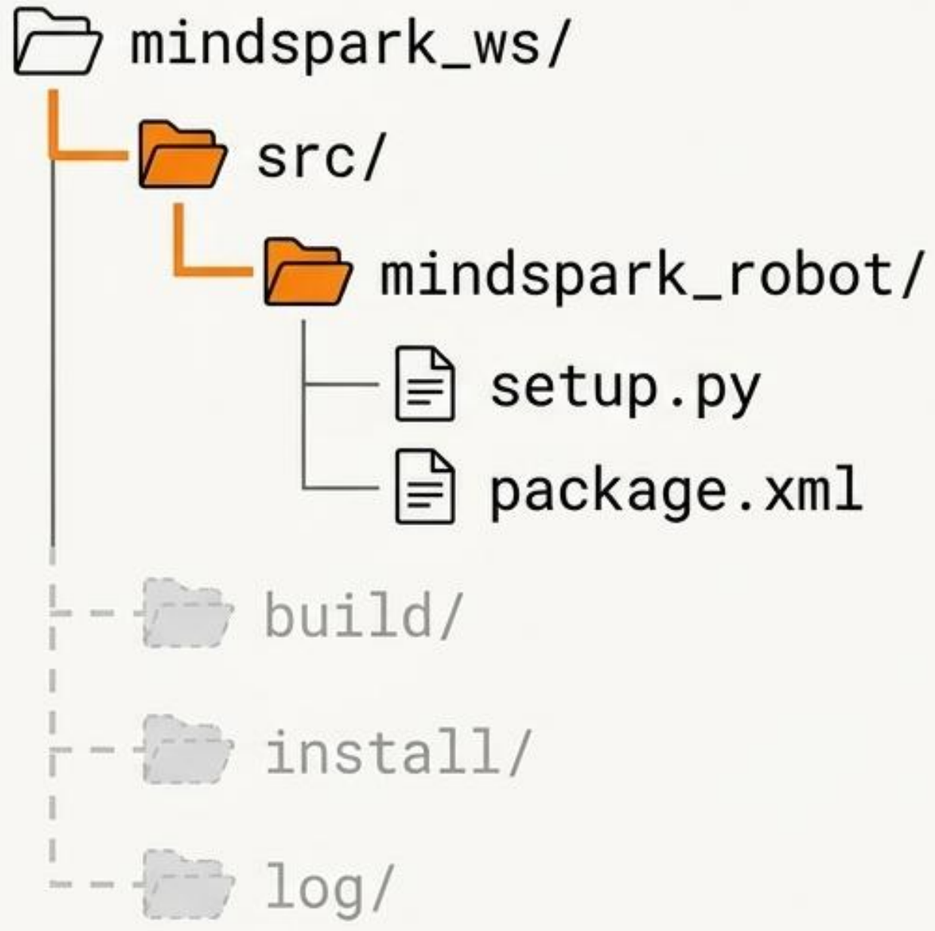
Expected State

Result: `~/mindspark_ws/src/mindspark_robot/` now exists with `setup.py` and `package.xml`.

Gemini Notebook

Anatomy of a ROS2 Workspace

Isolating human-written source code from auto-generated compiled binaries.



Human Workspace

Where you write code.
The only folder you should edit manually.

Colcon Workspace

Auto-generated dynamically during the build process. Do not edit manually.

Gemini Notebook

Task 3: Writing the First Node

~/mindspark_ws/src/mindspark_robot/mindspark_robot/hello_node.py

```
import rclpy
from rclpy.node import Node

class HelloNode(Node):
    def __init__(self):
        super().__init__('hello_node')
        self.timer = self.create_timer(1.0, self.callback)
    def callback(self):
        self.get_logger().info('MindSpark ROS2 Robot System Running!')

def main(args=None):
    rclpy.init(args=args)
    node = HelloNode()
    try:
        rclpy.spin(node)
    except KeyboardInterrupt: pass
    finally:
        node.destroy_node()
        if rclpy.ok():
if __name__ == '__main__':
    main()
```

Action: Write this into your IDE now. We will dissect the architecture on the next slide.

Gemini Notebook

Code Dissection: The Node Class

Architecture Blueprint

Attaches this script directly to the core ROS2 Middleware (RMW) backend.

```python
from rclpy.node import Node

class HelloNode(Node):
    def __init__(self):
        super().__init__('hello_node')
        self.timer = self.create_timer(1.0, self.callback)

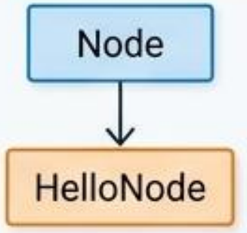
    def callback(self):
        self.get_logger().info('...')
```

Network Registration

`hello_node`

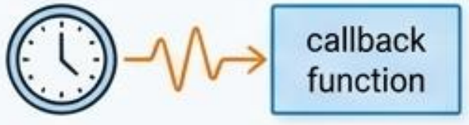
Registers the exact, unique name the active ROS2 network will recognize.

Visualizing Inheritance



HelloNode inherits directly from the base Node class.

The 1Hz Cyclic Pulse



Sets a 1.0-second interval timer that triggers the callback.

Gemini Notebook

Code Dissection: The Event Loop Execution

```
def main():
    rclpy.init(args=args)

    node = HelloNode()

    rclpy.spin(node)

    node.destroy_node()
    rclpy.shutdown()
}
```

Initialization: Starts the ROS2 communications backend.

Instantiation: Loads the custom class into active memory.



The Event Loop.
Thread execution physically blocks here. It spins indefinitely, listening for timer events or incoming network callbacks.

KeyboardInterrupt (Ctrl+C)

Teardown: The escape hatch. Safely flushes buffers and shuts down the network socket.

Gemini Notebook

Task 4: Executable Registration

Why we must edit setup.py to link ROS2 to Python.

(Action) **mindspark_robot/setup.py**

```
'hello_node = mindspark_robot.hello_node:main',
```

Registration makes the ROS2 system aware of your Python entry point ('main').

(Theory Execution Comparison)

Standard Python Execution

```
$ python3 hello_node.py
```

Requires absolute file paths to locate the script.

Possesses absolutely no ROS2 environment awareness or workspace integration.

ROS2 Integrated Execution

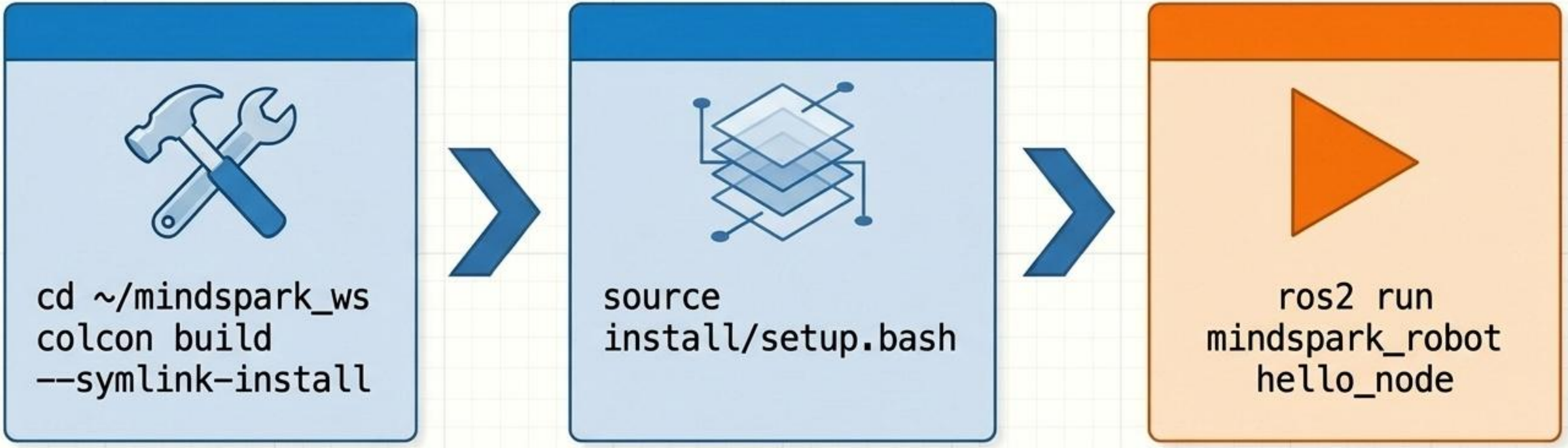
```
$ ros2 run mindspark_robot hello_node
```

Uses workspace environment variables.

Locates the executable package automatically and can be launched securely from anywhere in the file system.

Gemini Notebook

Task 5: The Build & Run Pipeline




```
[INFO] [hello_node]: MindSpark ROS2 Robot System Running!
[INFO] [hello_node]: MindSpark ROS2 Robot System Running!
[INFO] [hello_node]: MindSpark ROS2 Robot System Running!
```

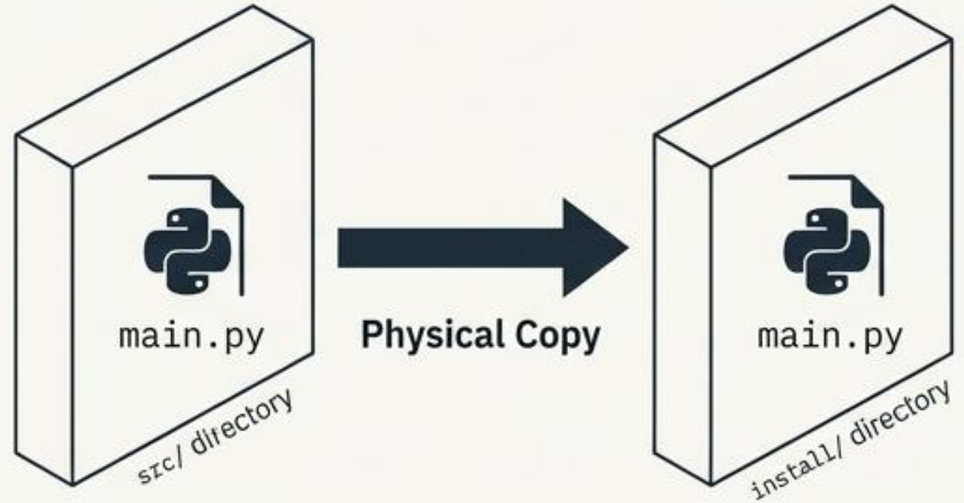
Keep this terminal running for the next task.

Gemini Notebook

Mechanics of a Symlink Install

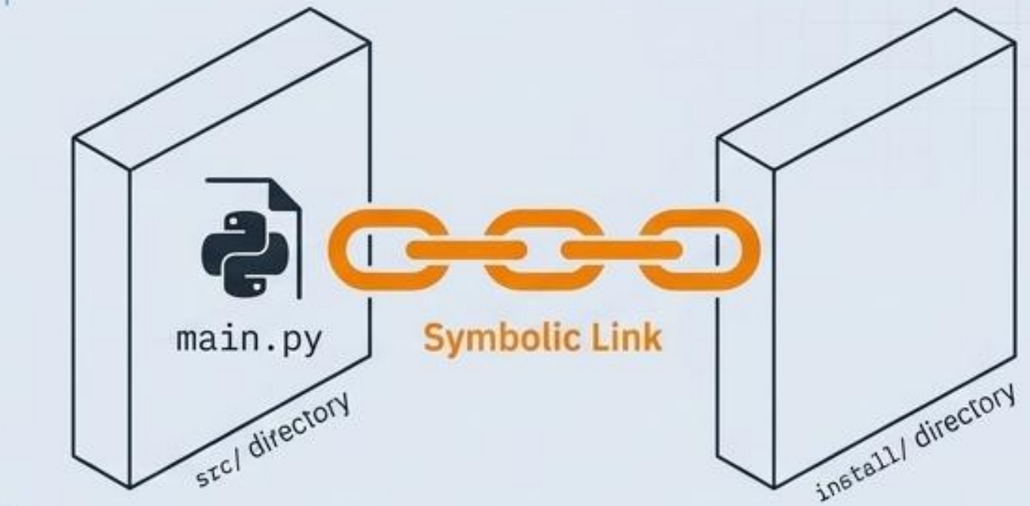
Why `--symlink-install` is critical for rapid Python development.

Default Build



Requires running '`colcon build`' entirely from scratch **every single time** you change a line of code.

Symlink Install



Creates a **symbolic link**. Changes saved in the human-facing `src/` folder are **instantly** and **dynamically** reflected in the active executable without ever needing to rebuild.

Gemini Notebook

Task 6: System Introspection

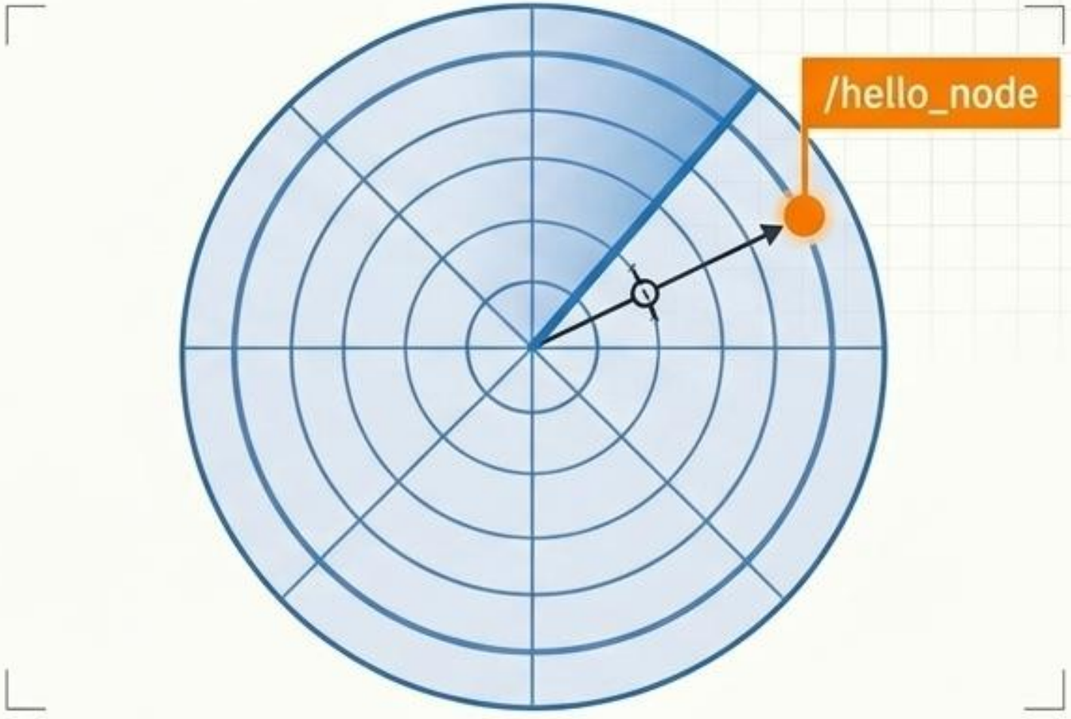
New Terminal

```
$ source /opt/ros/jazzy/setup.bash
$ source ~/mindspark_ws/install/setup.bash

$ ros2 node list
$ ros2 node info /hello_node
```

(Press **Ctrl+C** in Terminal 1 when finished to stop the node).

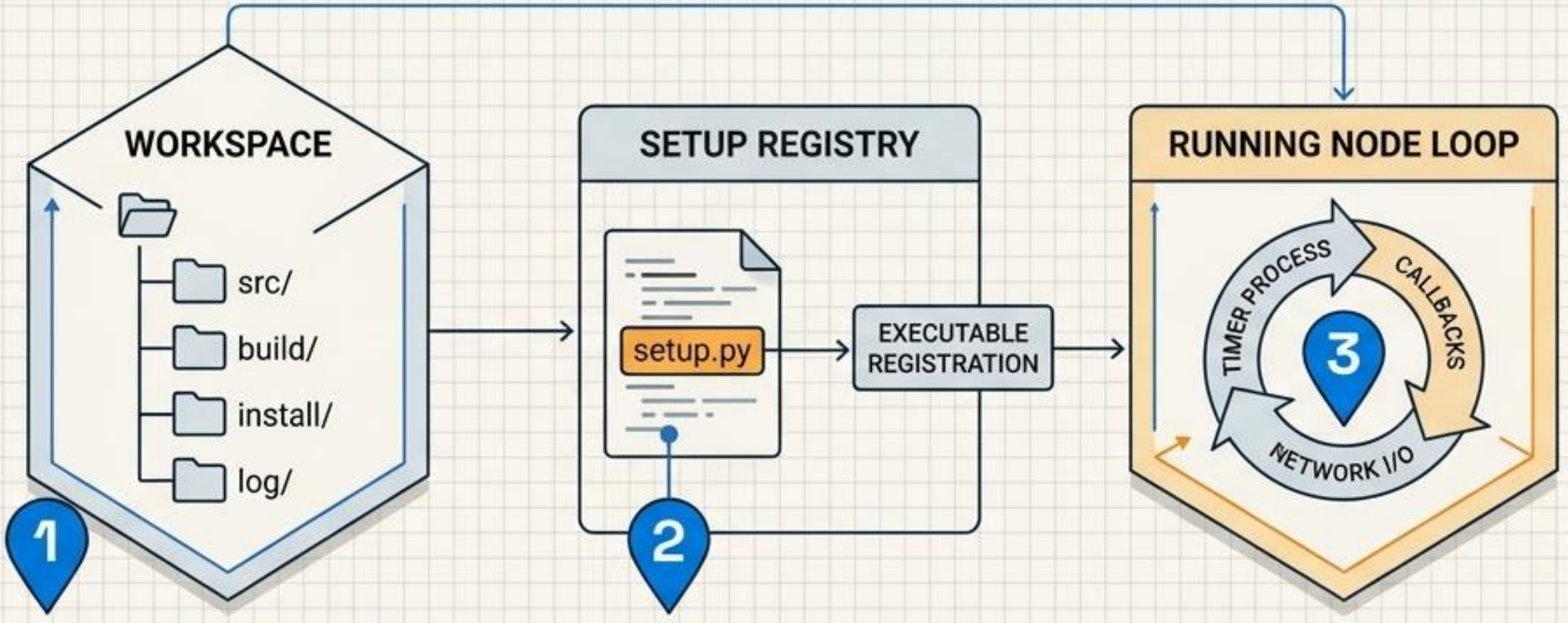
Network Radar



Independent terminals must physically 'join' the environment by sourcing the overlay. Only then can the daemon detect and interact with nodes running in the background.

Gemini Notebook

Lab Verification & Capstone Synthesis



Q1. Workspace Purpose: An isolated directory environment to build, manage, and overlay custom ROS2 packages without breaking the system underlay.

Q2. Registration: setup.py registers the Python node to the ros2 run CLI tools via the console_scripts array.

Q3. Execution: rclpy.spin(node) blocks the main thread, keeping the node alive indefinitely to continuously process timers and network callbacks.

EVIDENCE CHECKLIST

- [x] Distro is Jazzy
- [x] Package Created
- [x] Build Completed
- [x] Node Logs 1Hz
- [x] Node Visible in List

Gemini Notebook